

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 2: Verification of Timed Automata using UPPAAL

Prof. Daniel Holcomb

Submitted by: Shu-Lin Shih and Jaideep Khare
Student ID: 35199856 and 35193853
Lab No.: 2
Submission Date: March 2026

Overview

This report presents model construction, simulation, and symbolic verification for Lab 2 using UPPAAL. Part A analyzes timed automata for the Viking bridge-crossing problem under increasingly complex scenarios. Part B develops and verifies a timed traffic-light and pedestrian-control system, including safety analysis and a robustness extension required for ECE 622.

The workflow used in all sections was: (1) encode model changes in UPPAAL templates/system declarations, (2) state explicit verification queries, (3) run verifier checks, and (4) inspect symbolic or concrete traces to extract schedules and explain results.

1 Part A: Bridge-Crossing Timed Automata

A.1: Four Vikings within 60 minutes

Determine whether all four Vikings with crossing times 5 min, 10 min, 20 min, and 25 min can reach the safe side within 60 min, and provide a valid schedule.

Model checking query:

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && time <= 60
```

The query is satisfiable, so a feasible execution exists.

Witness schedule from trace:

#	Trip (Vikings by index; times in minutes)	Start time (min)	Trip origin
1	Vik ₁ + Vik ₂ cross to safe side (5+10)	0	Unsafe side
2	Vik ₁ returns with torch (5)	10	Safe side
3	Vik ₃ + Vik ₄ cross to safe side (20+25)	15	Unsafe side
4	Vik ₂ returns with torch (10)	40	Safe side
5	Vik ₁ + Vik ₂ cross to safe side (5+10)	50	Unsafe side

Table 1: A.1 crossing schedule (total completion time: 60 min).

The schedule completes all crossings at minute 60, satisfying the time bound exactly.

E> Viking1.safe and Viking2.safe and Viking3.safe and Viking4.safe and time <=60

Figure 1: A.1: UPPAAL verifier for bounded reachability within 60 min (displayed at $1.5\times$ prior proof scale, width $\leq$ text width).

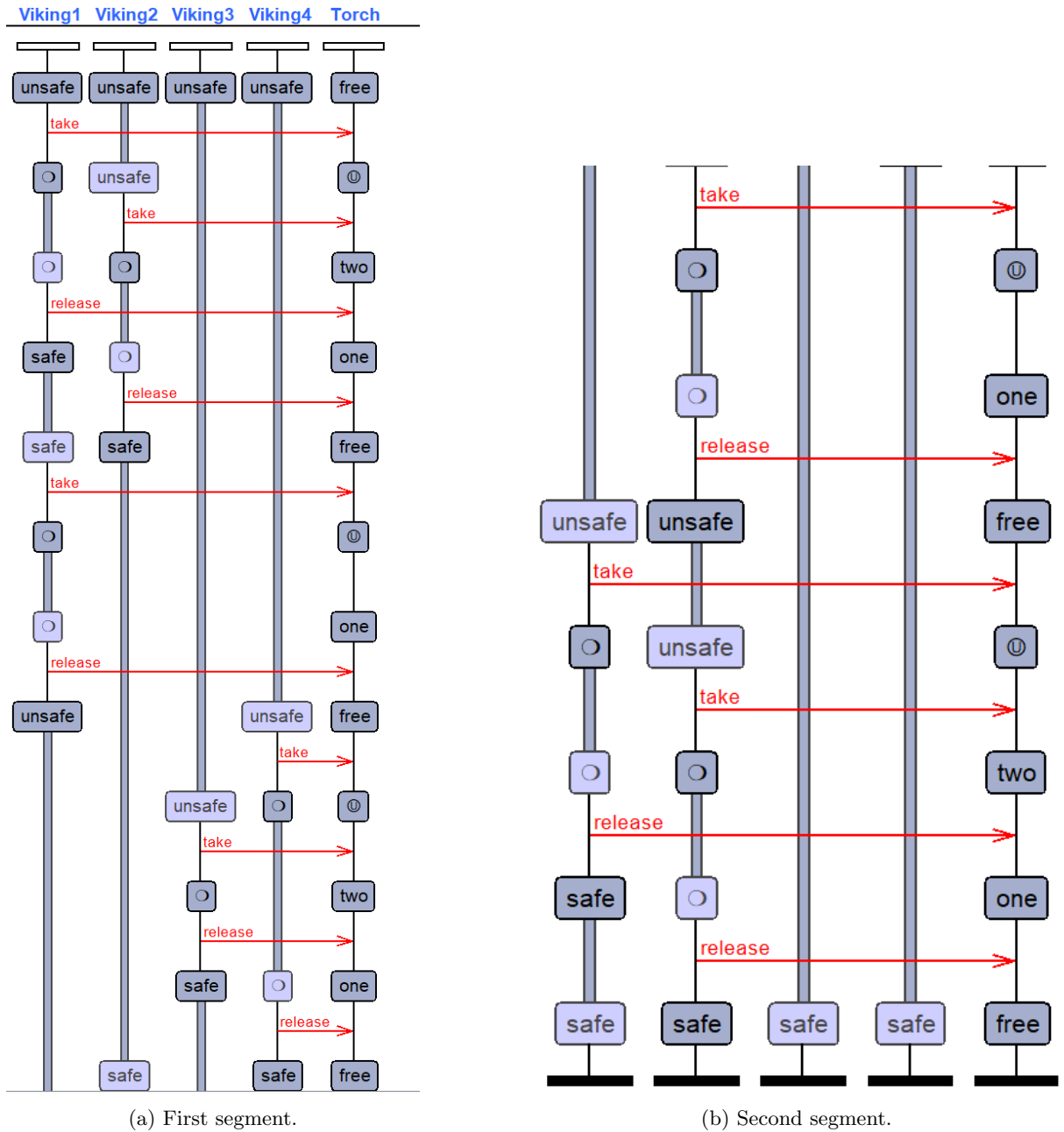


Figure 2: A.1: UPPAAL verifier/trace views for bounded reachability within 60 min (two panels).

A.2: Add a fifth Viking (10 minutes)

A fifth Viking is added with a 10 min bridge delay (in addition to the existing 10 min Viking). What is the shortest time for all five Vikings to reach the safe side? Describe model changes and queries used.

The fifth Viking is added as a separate template instance with delay `slowest2=10`, composed with the existing four Vikings and the torch:

```
const int slowest2 = 10;
Viking5 = Soldier(slowest2);
system Viking1, Viking2, Viking3, Viking4, Viking5, Torch;
```

Core reachability query (bounded completion time T):

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && Vik_5.safe && time <= T
```

A decreasing search over T and simulator tracing support the schedule below. In the optimal trace, Vik₁ and Vik₂ cross first and Vik₁ returns; then Vik₁ and Vik₅ cross and Vik₁ returns; next the slow pair Vik₃ and Vik₄

crosses together; Vik₂ returns with the torch; and the final trip is Vik₁ and Vik₂. The shortest completion time obtained for this model is 75 min.

#	Trip (Vikings by index; times in minutes)	Start time (min)	Trip origin
1	Vik ₁ + Vik ₂ cross to safe side (5+10)	0	Unsafe side
2	Vik ₁ returns with torch (5)	10	Safe side
3	Vik ₁ + Vik ₅ cross to safe side (5+10)	15	Unsafe side
4	Vik ₁ returns with torch (5)	25	Safe side
5	Vik ₃ + Vik ₄ cross to safe side (20+25)	30	Unsafe side
6	Vik ₂ returns with torch (10)	55	Safe side
7	Vik ₁ + Vik ₂ cross to safe side (5+10)	65	Unsafe side

Table 2: A.2 crossing schedule (witness trace; total completion 75 min).

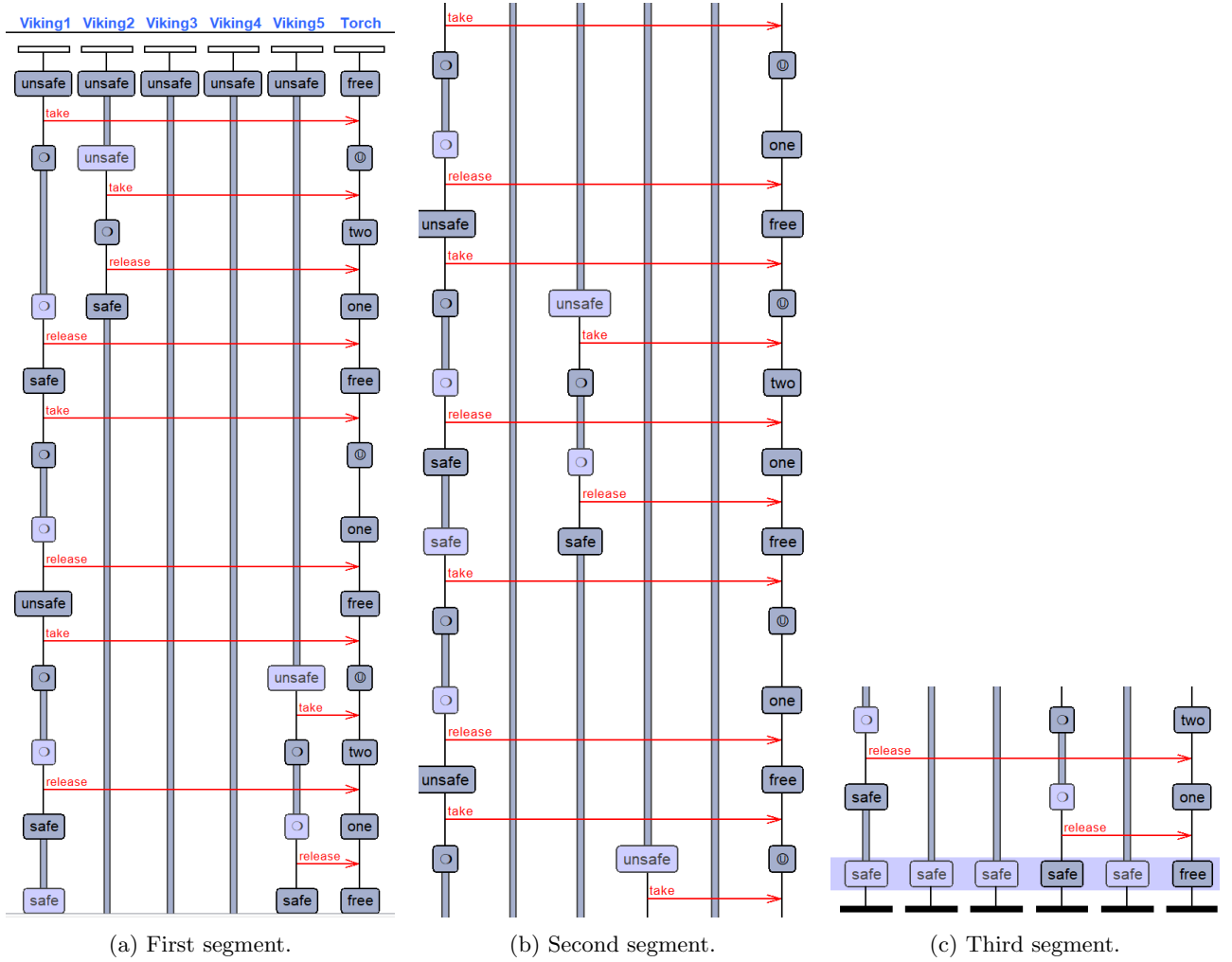


Figure 3: A.2: UPPAAL simulator trace split across three views.

A.3: Two-bridge scenario

Building on A.2, five Vikings may choose between two bridges: the original bridge (capacity 2, normal crossing delay) and a new bridge (capacity 3, each Viking takes $2 \times$ their original bridge delay). Both bridges require the same torch, so only one bridge can be in use at a time. Determine the shortest completion time and provide a schedule with a fourth column indicating original vs. new bridge.

We updated the model to match the official two-bridge statement. The specific UPPAAL changes are:

- In the Torch template, an additional path was added so up to three Vikings can take the torch for the new-bridge crossing case.

- An urgent location in the Torch template forces this three-Viking path to execute without time elapse at that decision point.
- In the Viking (Soldier) template, a second release path was added with a different delay on the release synchronization. This encodes the per-path timing difference while still merging at the same destination location.

The query evidence for the time bound is:

```
E<> Viking1.safe && Viking2.safe && Viking3.safe && Viking4.safe && Viking5.safe && time <= 70
```

Case	Bound	Result
1	<code>time <= 70</code>	Not satisfied (no execution reaches all five safe within 70 seconds).
2	<code>time <= 75</code>	Satisfied (feasible witness trace exists).

Table 3: A.3 bounded-reachability check: 75 seconds is feasible, 70 seconds is not.

#	Trip (Vikings by index; times in seconds)	Start time (s)	Trip origin	Bridge used
1	Vik ₁ + Vik ₂ cross to safe side (5+10)	0	Unsafe side	Original
2	Vik ₁ returns with torch (5)	10	Safe side	Original
3	Vik ₁ + Vik ₅ cross to safe side (5+10)	15	Unsafe side	Original
4	Vik ₁ returns with torch (5)	25	Safe side	Original
5	Vik ₃ + Vik ₄ cross to safe side (20+25)	30	Unsafe side	Original
6	Vik ₂ returns with torch (10)	55	Safe side	Original
7	Vik ₁ + Vik ₂ cross to safe side (5+10)	65	Unsafe side	Original

Table 4: A.3 witness schedule reaching completion at 75 seconds. (Simulation screenshots omitted by request.)

2 Part B: Traffic-Light and Crosswalk Verification

B.1: Discrete-model reasoning (before timed implementation)

For each property, a reachability-oriented check was defined conceptually and then evaluated by model intuition over the discrete behavior (no tool reachability on the discrete model in this step).

Prop.	Reachability check idea	Expected truth	Rationale (intuition)
(a)	It is impossible for the traffic light to be in the pending state 70 s after the initial condition.	False	Reachable if the pedestrian button is pressed shortly after entering green, so pending can still be active at 70 s.
(b)	It is possible for the traffic and crosswalk lights to be green at the same time.	False	The pedestrian side exits green at around 55 s, while the traffic side needs at least 60 s before returning to green, so overlap is excluded in this discrete intuition.
(c)	The traffic light will never stay in the green (non-pending) location longer than 120 s.	True	The location invariant forces a jump by 120 s; the controller cannot remain green beyond that bound.
(d)	The traffic light will never stay in the pending location for 45 s or longer.	False	If <code>pedButton</code> is pressed immediately at green entry, pending can persist beyond 45 s.
(e)	The traffic light will never stay in the pending location for 65 s or longer.	True	Pending can extend to about 60 s in worst case, but not to 65 s.
(f)	It is possible that the crosswalk is red while the traffic light is green.	True	This is a normal reachable operating phase of the controller pair.
(g)	Whenever the crosswalk is red, the traffic light must be green.	False	Counterexample: yellow traffic with red crosswalk is reachable.
(h)	The 3rd occurrence of <code>sigG</code> must not happen within the first 300 s after the initial condition.	True	With approximate period 125 s (without extra pedestrian stall), only two green events should occur before 300 s.
(i)	The 3rd occurrence of <code>sigG</code> must not happen within the first 400 s after the initial condition.	False	By around 375 s, repeated cycles allow multiple green events; the statement does not hold at 400 s.

Table 5: B.1 qualitative checks and expected outcomes (intuition only).

B.2: Timed automata model construction

The timed UPPAAL model includes:

- Traffic-light and crosswalk-light automata connected by broadcast/synchronization channels (e.g., `sigR`, `sigG`, `sigY`, `pedG`, `pedR`).
- Non-deterministic pedestrian input trigger (`pedinput`): implemented as a self-looping transmitter so that the pedestrian button channel can fire concurrently with other synchronizations. The self-loop does not encode meaningful defaulting behavior; it exists so that UPPAAL's channel semantics allow the input to be offered in the same step as other transitions when needed.
- Receiver/sink automata for outputs that would otherwise lack a listener: unused outputs are connected to simple self-looping receivers that absorb signals, which keeps broadcast sends from blocking the rest of the model.
- Urgent intermediate locations where needed to split transitions that conceptually combine input and output events.

The model was exercised with concrete simulation and simple reachability checks before formal verification in B.3.

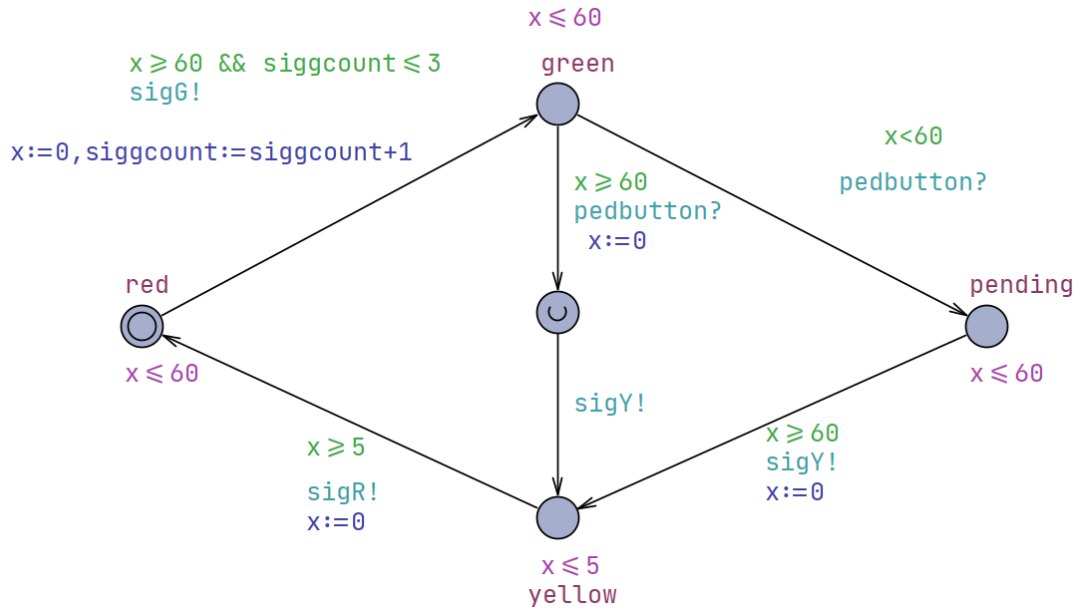


Figure 4: Timed automata for the traffic-light system (B.2).

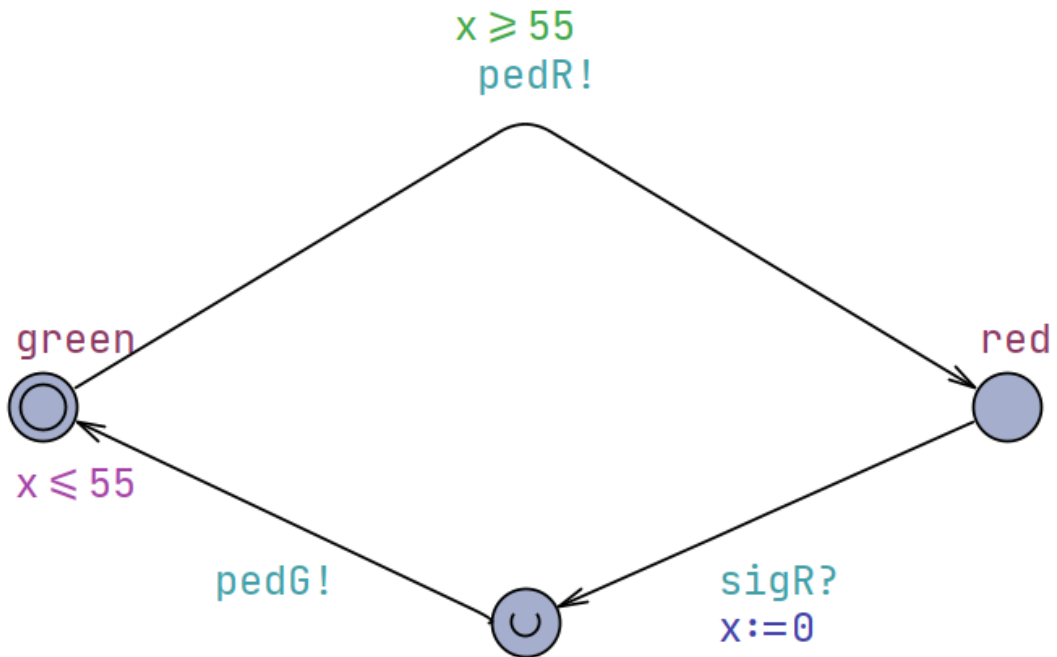


Figure 5: Crosswalk / pedestrian-side signal system (B.2).

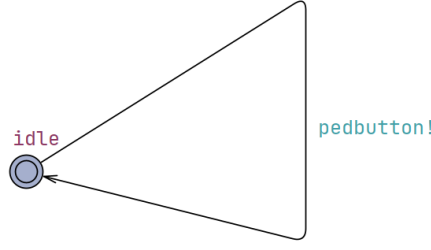


Figure 6: Non-deterministic `pedinput` trigger (self-loop for channel firing). Image width is half of the first two B.2 figures.

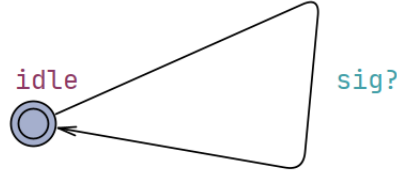


Figure 7: Receiver/sink automata for unused outputs. Image width is half of the first two B.2 figures.

```
E< crosswalklight1.green
E< crosswalklight1.red
E< trafficlightcontroller1.pending
E< trafficlightcontroller1.green
E< trafficlightcontroller1.yellow
A[] not deadlock
```



Figure 8: Reachability / sanity checks used while testing the B.2 model.

B.3: Query mapping and verification outcomes

Each B.1 English statement was translated into a UPPAAL query over the B.2 timed model. The verifier panel confirms results for all checks in one model snapshot.

Prop.	Representative UPPAAL query form	UPPAAL outcome	Final statement truth
(a)	<code>E<> (globalTime==70 && traffic.pending)</code>	Reachable	False
(b)	<code>E<> (traffic.green && crosswalk.green)</code>	Reachable (timed model)	False
(c)	<code>A[] (traffic.green imply tGreen<120) and E<> (traffic.green && tGreen>=120)</code>	Satisfied / Not reachable	True
(d)	<code>A[] not (traffic.pending && tPend>=45)</code>	Violated	False
(e)	<code>A[] not (traffic.pending && tPend>=65)</code>	Satisfied	True
(f)	<code>E<> (traffic.green && crosswalk.red)</code>	Reachable	True
(g)	<code>A[] (crosswalk.red imply traffic.green)</code>	Violated	False
(h)	<code>A[] (sigGCount<3 or globalTime>300)</code>	Satisfied (target interpretation)	True
(i)	<code>A[] (sigGCount<3 or globalTime>400)</code>	Violated	False

Table 6: B.3 query mapping with verifier outcomes and final truth values for the original English statements.

Note. Some formulas are written as reachability checks of counterexamples. Therefore, a query outcome and the final truth of the original English statement can be opposite by design.

```

//i
E> siggcount ==3 and globaltime ≤ 400
//h
E> siggcount ==3 and globaltime ≤ 300
//g
E> trafficlightcontroller1.yellow and crosswalklight1.red
E> trafficlightcontroller1.red and crosswalklight1.red
//f
E> trafficlightcontroller1.green and crosswalklight1.red
//e
E> trafficlightcontroller1.pending and trafficlightcontroller1.x >65
//d
E> trafficlightcontroller1.pending and trafficlightcontroller1.x >45
//c
E> trafficlightcontroller1.green and trafficlightcontroller1.x >90
//b
E> trafficlightcontroller1.green and crosswalklight1.green
//a
E> trafficlightcontroller1.pending and globaltime = 70

```

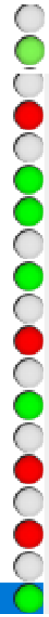


Figure 9: Verifier window: B.3 queries with pass/fail indicators.

B.4: Pedestrian component and unsafe crossing-time range

The previous button-generator and `pedG` receiver were replaced with an explicit pedestrian automaton having three locations: *Idle*, *Waiting*, and *Crossing*. Parameter `pedCrossTime` controls required crossing time.

Bad condition: the system is unsafe when traffic becomes green while the pedestrian remains in the crossing location.

Observed threshold: reachability checks show the bad condition is unreachable at `pedCrossTime=59` but reachable at `pedCrossTime=60`. The smallest unsafe value is therefore 60, and values ≥ 60 are unsafe for this model.

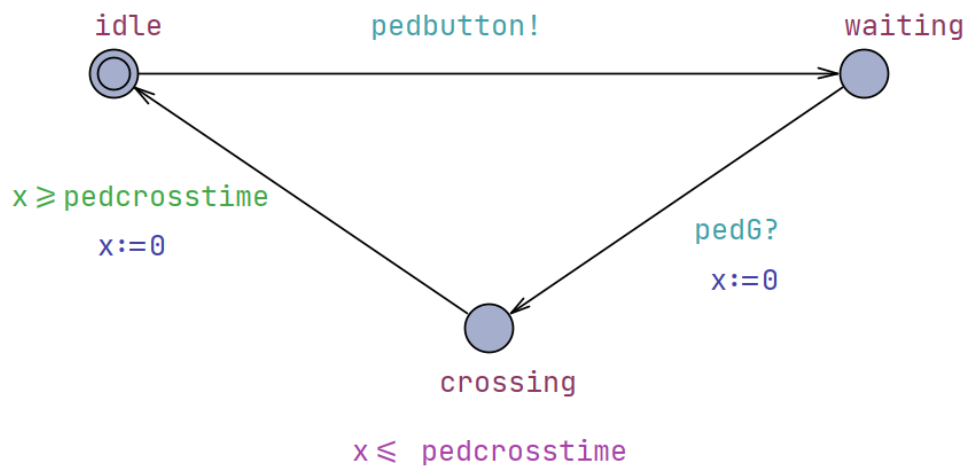


Figure 10: Pedestrian template with `pedCrossTime` and crossing-state timing (B.4).

```

// receiver pedG = receiver (ped
pedstrain1=pedstrain(59);

```

E> pedstrain1.crossing and trafficlightcontroller1.green



Figure 11: B.4 safe case at `pedCrossTime=59`: query/clause (top) and trace (bottom).

```
pedestrian1=pedestrian(60);
```

```
E> pedestrian1.crossing and trafficlightcontroller1.green
```

Figure 12: B.4 unsafe case at `pedCrossTime=60`: query/clause (top) and trace (bottom).

B.5 (ECE 622): Robustness change for arbitrary crossing time

Lab 2 asks for a more robust model such that safety holds regardless of how long the pedestrian takes to cross; explain the change; repeat the B.4 safety query at a `pedCrossTime` value that was unsafe before and show it is safe after the change; submit `_b5.xml` with the report.

To remove dependence on a bounded pedestrian crossing duration, a synchronization signal `done` was introduced from the pedestrian automaton to the traffic controller. The controller defers returning to green until the pedestrian reports crossing completion.

The previously unsafe test point from B.4 (`pedCrossTime=60`) becomes safe after adding the `done` handshake and updating controller guards accordingly.

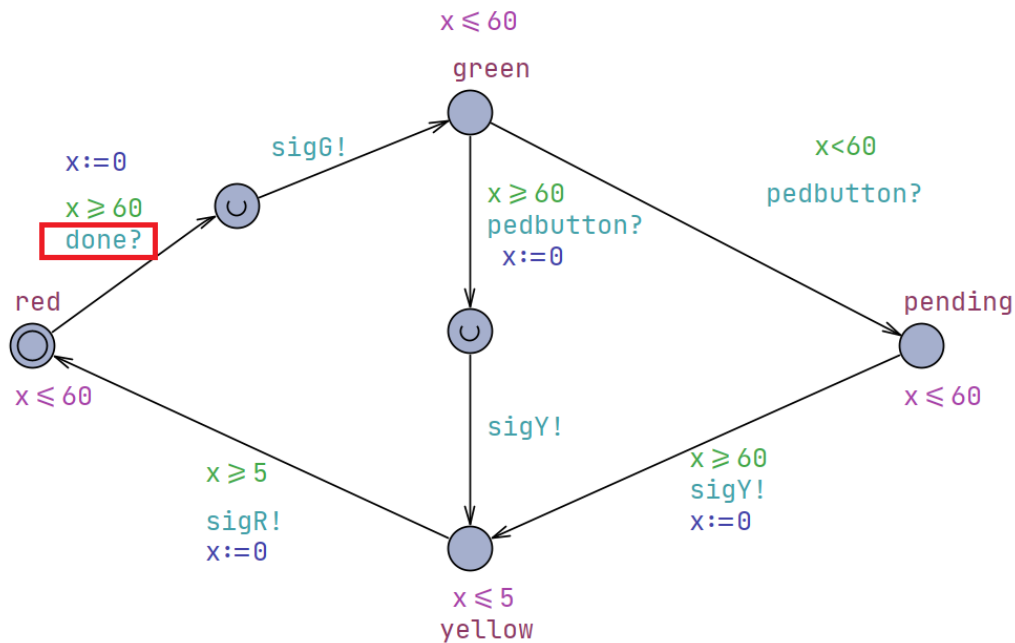


Figure 13: Traffic controller with `done` handshake (B.5).

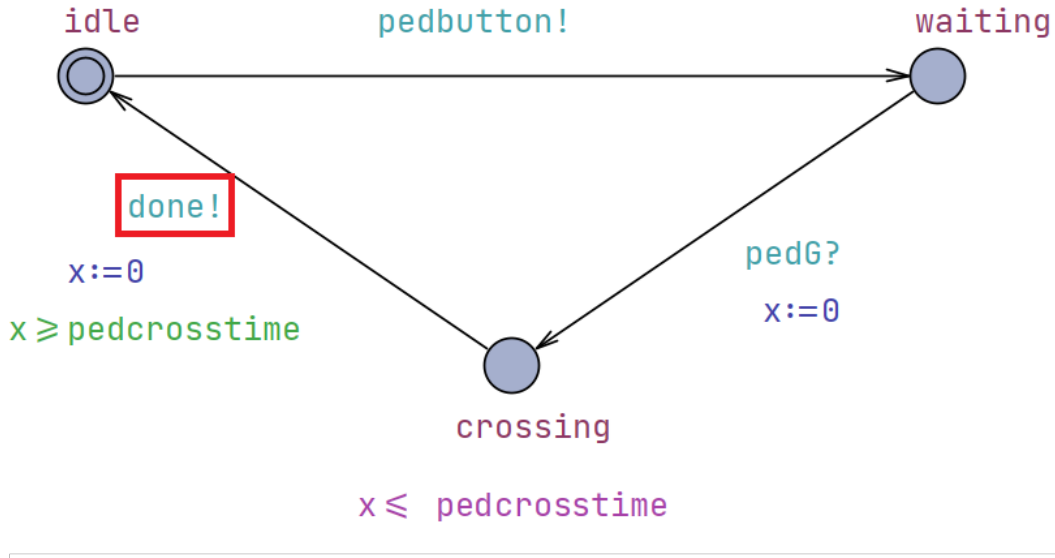


Figure 14: Pedestrian model cooperating with **done** (B.5).

```
E> pedestrain1.crossing and trafficlightcontroller1.green
```

Figure 15: B.5: safety re-check at `pedCrossTime=60` (now safe).

```
pedestrain1=pedestrain(60);
```

Figure 16: B.5: corresponding query/clause for the unsafe condition at `pedCrossTime=60`.

```
E> Viking1.safe and Viking2.safe and Viking3.safe and Viking4.safe and time ≤ 60
```

Figure 17: B.5: additional confidence checks (e.g. crossing allowed under robust model).

#	Lab 2 B.5 requirement (paraphrase)	What this report demonstrates	OK?
1	Robust model: safe regardless of crossing duration.	done handshake; green deferred until crossing completes (Figs. 13–14).	Y
2	Explain the change in the report.	Text and figures; done removes reliance on a fixed <code>pedCrossTime</code> for safety.	Y
3	Repeat B.4 safety query at a previously unsafe <code>pedCrossTime</code> ; show safe after change.	Figs. 15–16: bad condition unreachable at <code>pedCrossTime=60</code> .	Y
4	Simulate / use extra queries for confidence.	Fig. 17 and B.2-style checks.	Y
5	Submit <code>_b5.xml</code> with the report.	Course-style filename (e.g. <code>*_b5.xml</code>) included with submission materials.	Y

Table 7: B.5 alignment checklist: lab wording vs. evidence in this report (for grading).

Conclusion

This lab demonstrates how UPPAAL supports rigorous analysis of both scheduling and reactive control problems. In Part A, bounded reachability and trace extraction were used to derive crossing schedules under increasingly rich constraints. In Part B, timed automata and formal queries were used to relate informal predictions to tool results, to expose unsafe regimes, and to validate a robustness refinement that removes dependence on a fixed pedestrian timing assumption.

Appendix: Tooling and Submission Artifacts

This report was prepared using Cursor AI and compiled with `latexmk` on the VLSICAD server, using the lab project working directory as the build root.

Final submission model files

- `shih_a2.xml`
- `shih_a3.xml`
- `shih_b3.xml`
- `shih_b4.xml`
- `shih_b5.xml`

Repository link

https://github.com/JDKhare/UMass_ECE654_Projects